



FUZZ TESTING

Find bugs in inputs
you never imagined.



```
""  
A  
AA  
123  
-100  
😊  
こんにちは  
\x00\x01\x02  
VeryLongString...  
InvalidUTF8...  
???  
...and thousands more!
```



CHAPTER
7

GO TESTING HANDBOOK



YOUR TESTS ONLY COVER WHAT YOU EXPECT.



Unit tests verify **predefined inputs**.



But **unexpected inputs**
can still break your code.



What happens when
the input **surprises you**?

EXPECTED INPUTS

```
tests := []struct {  
    input    string  
    expected string  
}{  
    {"Go", "oG"},  
    {"Hello", "olleH"},  
    {"", ""},  
    // ...  
}
```



Covered by
your tests



UNEXPECTED INPUTS

- "" (empty string)
- 🤪 (emoji)
- \x00\x01\x02 (invalid bytes)
- VeryLongString.... (extremely long)
- ... (and many more)



May break
your code



Good tests are important.
But they **can't predict everything**.



LET GO EXPLORE THE UNKNOWN.

Go automatically generates and mutates
thousands of inputs to discover:



PANICS

Find crashes and panics.



UNEXPECTED BEHAVIOR

Uncover logic issues you didn't anticipate.



EDGE CASES

Hit rare, unusual, and extreme inputs.



POTENTIAL VULNERABILITIES

Surface security issues and weak spots.



Fuzzing doesn't replace your tests.
It finds what your tests can't see.



MEET FUZZ()

```
func FuzzReverse(f *testing.F) {  
    f.Add("Go")  
  
    f.Fuzz(func(t *testing.T, input string) {  
        Reverse(input)  
    })  
}
```



SEED



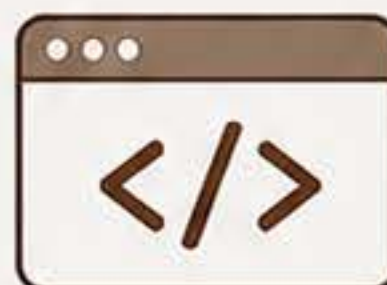
You provide
initial inputs.

GENERATE



Go generates and
mutates thousands
of variations.

EXECUTE



Your code runs
against each
generated input.

DISCOVER



Go finds crashes,
panics, and unexpected
behavior.



Fuzz tests let Go explore what you didn't think to test.
More inputs. More exploration. More confidence.



START WITH GOOD SEEDS.

FIND UNEXPECTED FAILURES.

GOOD SEED INPUTS

""	""
Go	"Go"
😊	"😊"
こんにちは	"こんにちは"
Aa...Aa	VeryLongString
\x00\x01	InvalidData



GO MUTATES THESE INPUTS
and explores new possibilities.



FAILURE FOUND?

Go preserves the input
so you can reproduce it.

FAILURE OCCURS



Panic / Crash /
Unexpected Behavior

INPUT PRESERVED



Failing input
is saved.

REPRODUCIBLE



Run again.
Same input.
Same failure.



A randomly found failure
becomes a permanent regression test.



DON'T TEST EVERY INPUT.

TEST WHAT MUST ALWAYS BE TRUE.

INSTEAD OF:

"Go" → "oG"

Go



oG

Tests a single example.
Misses everything else.

THINK:

$\text{Reverse}(\text{Reverse}(x)) == x$

x



Reverse



Reverse



x

Tests a property (invariant).
Holds for all valid inputs.



"" Go abc こんにちは 😊 123

!@#

{ }

\x00\x01

∞

...

你好



Property holds



We don't care
which input it is.
We care that the
property always
holds.

**ONE INVARIANT.**

Focus on the rule
that must always hold.

**THOUSANDS OF
GENERATED INPUTS.**

Go explores the rest.
You gain confidence.



EXPLORE. DISCOVER. REPRODUCE.

```
go test -fuzz=Fuzz
```



EXPLORE

Go generates and mutates thousands of inputs.



DISCOVER

Find crashes, panics, and unexpected behaviors.



REPRODUCE

Failures are saved so you can replay and fix them.



USEFUL FOR:



Parsers



Protocols



Encoders / Decoders



Unicode



JSON & XML



Input validation



File formats



**STRONGER
& MORE RELIABLE
CODE**



**USE FUZZING WHERE
INPUTS ARE UNPREDICTABLE.**



Fuzzing won't replace your unit tests.
It complements them by **exploring** what you don't expect.



@go_evangelist

END

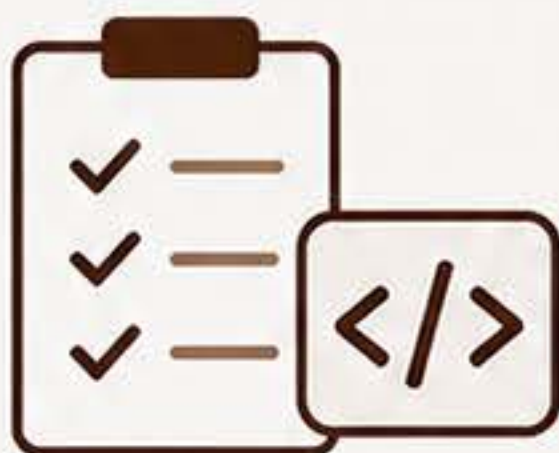


UNIT TESTS + FUZZ TESTS

=

STRONGER CONFIDENCE

UNIT TESTS



→ Known scenarios



→ Expected behavior



FUZZ TESTS



→ Unknown inputs



→ Unexpected behavior



DON'T TEST ONLY WHAT YOU EXPECT.
**TEST WHAT YOU
HAVEN'T IMAGINED YET.**



MORE COVERAGE
Explore more
of the unknown.



FIND HIDDEN BUGS
Discover issues
traditional tests miss.



REPRODUCIBLE FAILURES
Go saves failing inputs
so you can fix and prevent
regressions.



STRONGER SOFTWARE
Build resilient code
that handles the
unexpected.



Go Testing Handbook
Chapter 7: Fuzz Testing

Keep exploring.
Keep building reliable Go code.

